CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 1 Report

Team Members: _____Elliot White_____

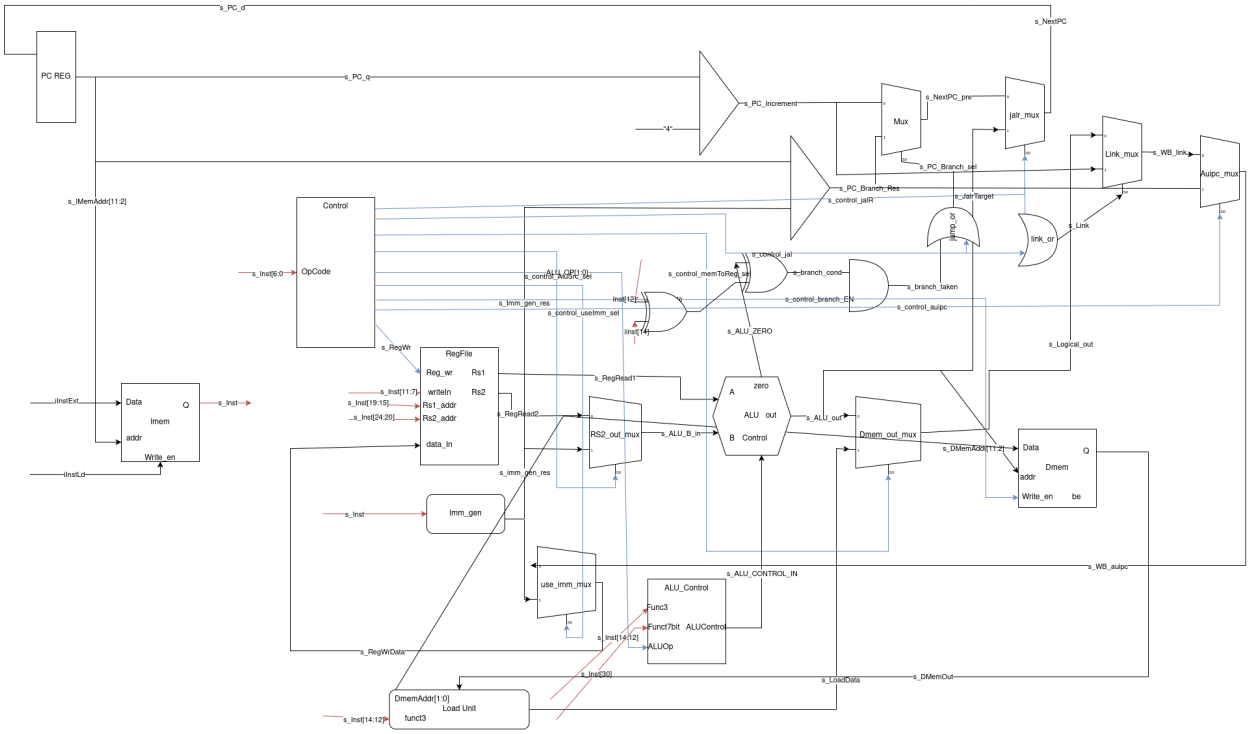
_Zach Sexton_____

_____

Project Teams Group #:__E_04_____

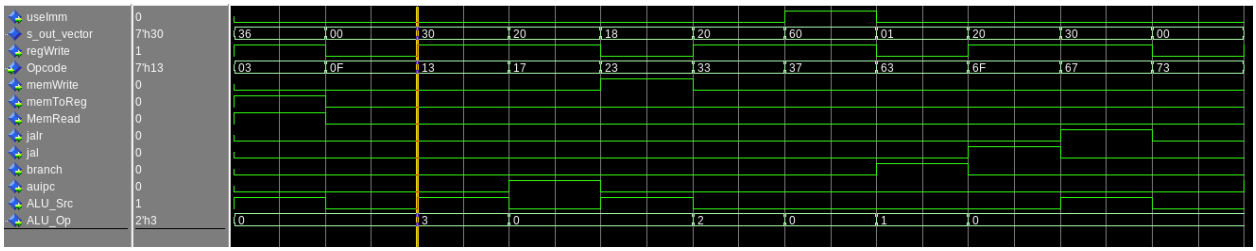
Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.



[Part 3.1.a.] Create a spreadsheet detailing the list of M instructions to be supported in your project alongside their binary opcodes and function fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the N control signals needed by your datapath implementation. The end result should be an N*M table where each row corresponds to the output of the control logic module for a given instruction. https://docs.google.com/spreadsheets/d/1VnQm_grHKPbwOLRKSL-QAsXKC9aNpEy76ClSIqWdhAc/edit?usp=sharing

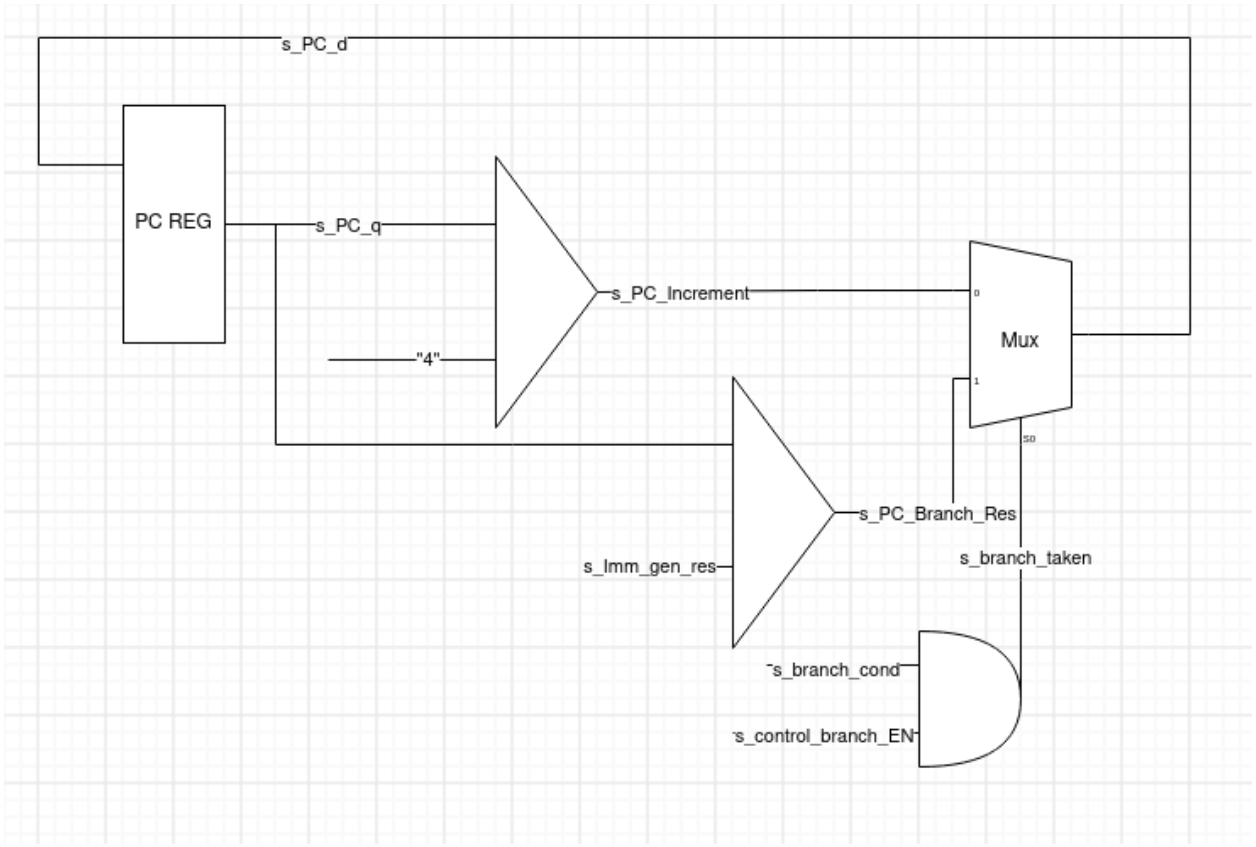
[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).



[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

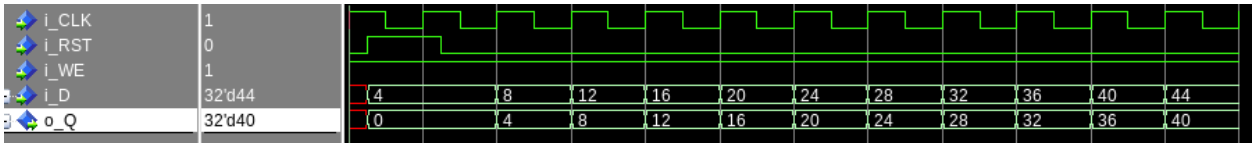
Branch instructions must be able to jump to arbitrary address, conditional based on an arithmetic operation. Therefore, an adder with the immediate from the instruction gives us the address capability, and a special invertible “zero” signal from the alu gives an efficient method to capture the results of the operation. By then disabling any write signals to memory or reg, we isolate the result of the comparison from our data.

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?

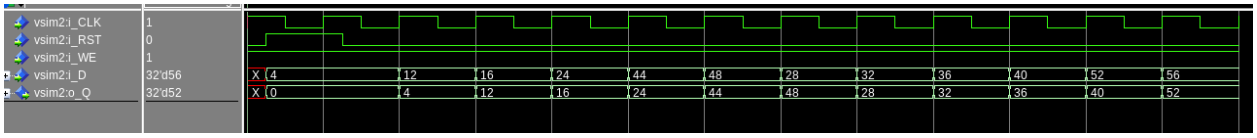


S_branch_cond originates from the invertible ALU_zero signal

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.



Test with no control flow, REG increases by 4 every clock cycle



Test with simplebranch.s, showing branch module working as expected

[Part 3.3.1.(a)] Describe the difference between logical (srl) and arithmetic (sra) shifts. Why does RISC-V not have a sla instruction?

A logical right shift (srl) moves every bit right and fills the vacated top bits with 0. An arithmetic right shift (sra) fills them with a copy of the original sign bit, so a negative number stays negative. For example, 0x80000000 >> 31 gives 0x00000001 logically and 0xFFFFFFFF arithmetically. Arithmetic right shift is effectively division by 2^n.

There is no sla because an arithmetic left shift would be identical to a logical left shift. Shifting left always fills the low bits with 0, and the sign bit is not preserved by either one, so sll covers both

[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.
barrel_shifter.vhd is a right shifter built from 5 cascaded mux2t1_N stages. Stage k shifts by 2^k (1, 2, 4, 8, 16) when shift-amount bit k is 1, and passes the data through unchanged when it is 0. Any shift from 0 to 31 is a combination of those stages. Every stage is wired the same way, so one generate loop builds all five.
Logical and arithmetic shifts differ only in the fill bit. The code computes s_Fill = i_Arith AND i_D(31) with an andg2. For srl, i_Arith is 0, so the fill is 0. For sra, the fill is the sign bit. Every stage uses that same fill bit for the bits shifted in at the top.

[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.
A left shift is a right shift of the bit-reversed value. The shifter reverses the input bits with a mux on i_Left, shifts right, then reverses the result back with a second mux on i_Left. No second shifter is needed. Left shifts are always logical, so i_Arith is 0 for them.
In the ALU, the control bits drive the shifter directly: i_Left is the inverse of ALUControl bit 0, and i_Arith is ALUControl bit 1. So SLL is 0100, SRL is 0101 and SRA is 0111.

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.





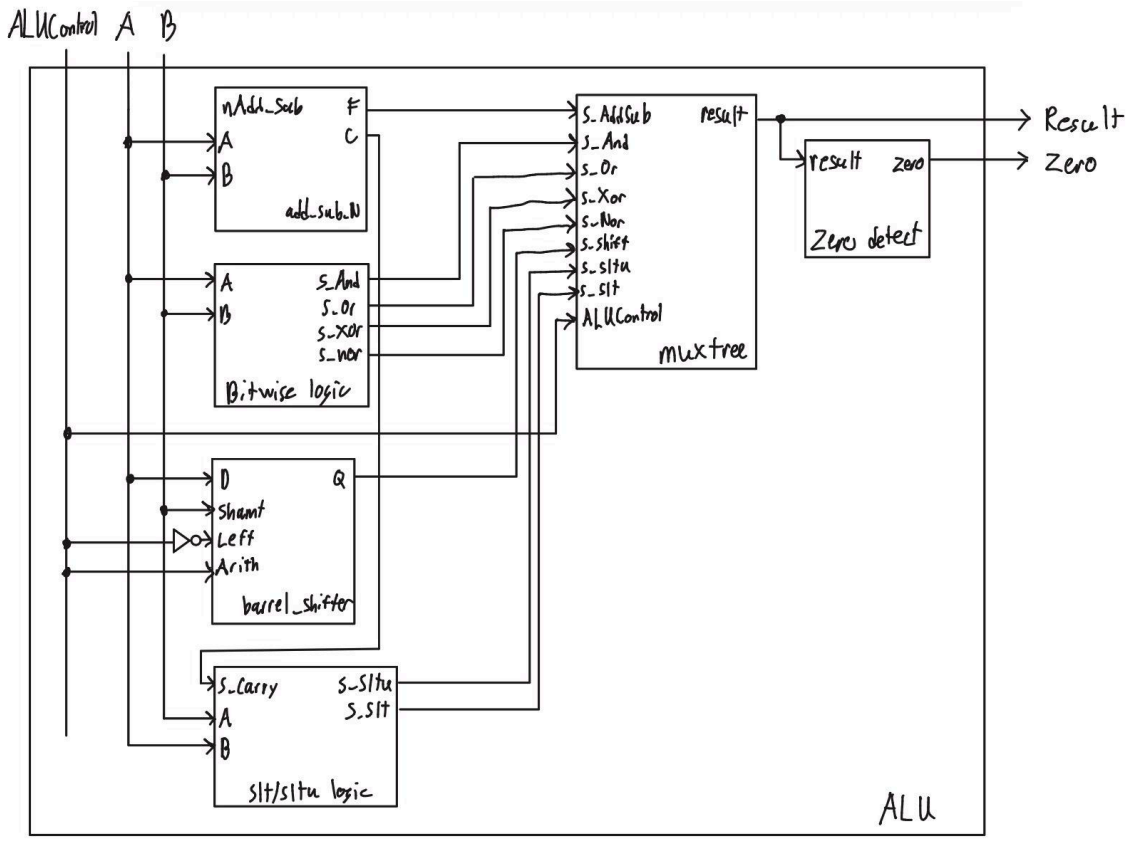
Case	Operation	Input	Shift	Expected	What it shows
1	sll	0x00000001	4	0x00000010	left shift
2	sll	0x00000001	31	0x80000000	largest left shift
3	srl	0x0000FF00	8	0x000000FF	logical right shift
4	srl	0x80000000	31	0x00000001	srl fills with 0
5	sra	0x80000000	31	0xFFFFFFFF	sra fills with the sign bit (1)
6	sra	0xF0000000	4	0xFF000000	sra of a negative number
7	srl	0xF0000000	4	0x0F000000	same input as 6, srl fills with 0
8	sra	0x7FFFFFFF	4	0x07FFFFFF	sra of a positive number fills 0
9	srl	0x12345678	0	0x12345678	shift by 0 changes nothing
10	sll	0x0000000F	16	0x000F0000	shift by 16 (last stage)

[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

We used the example processor discussed in lecture and the diagram shown in the lab document as a starting point. Our top level design was structural. Structural style was also used in most of the subcomponents, however mux-like devices, like the control modules and the immediate generator were dataflow.

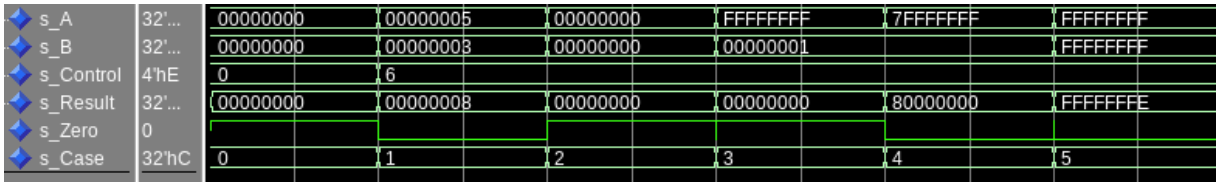
[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

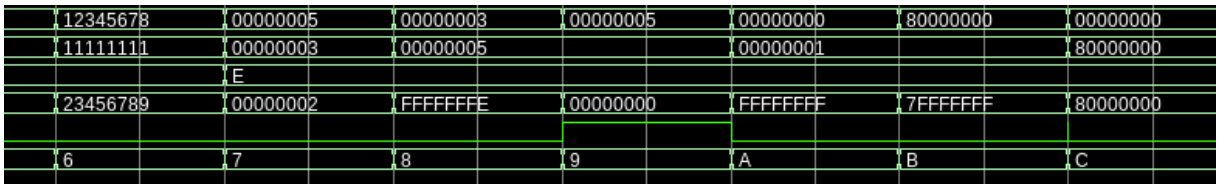
[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is slt implemented?

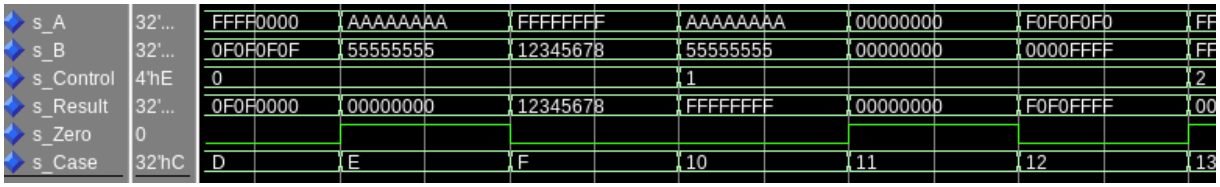


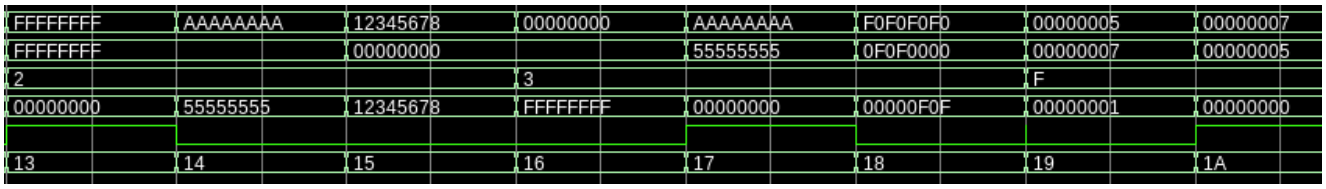
For calculating zero we use 31 2 input or gates in 5 levels to determine if the result is zero. We then use an inverter to get output = 1 if the result was zero and output = 0 if the result was not zero. Slt is a subtraction that checks whether the result would be negative. When the ALUControl bit 3 is 1, the adder computes a + ~b + 1, which is a - b. We reuse the adder's cout instead of the sign of the result, because the sign of a - b is wrong when the subtraction overflows

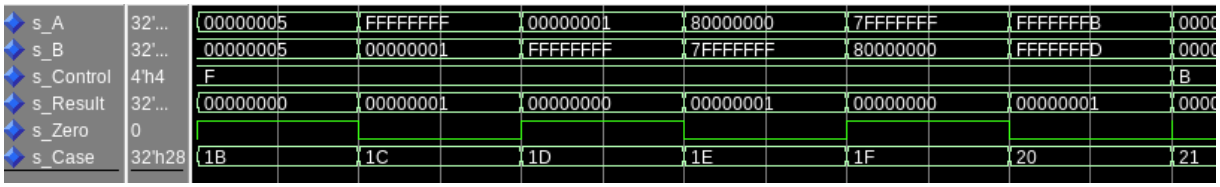
[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

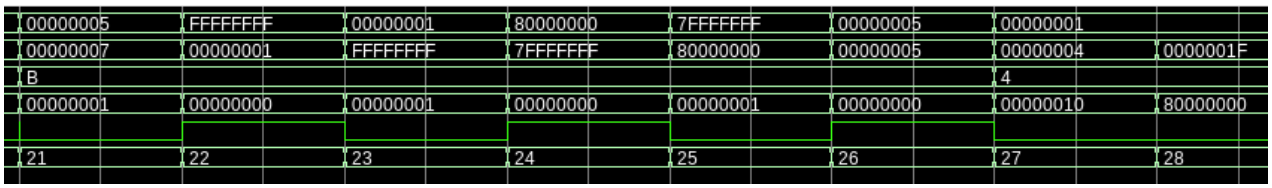


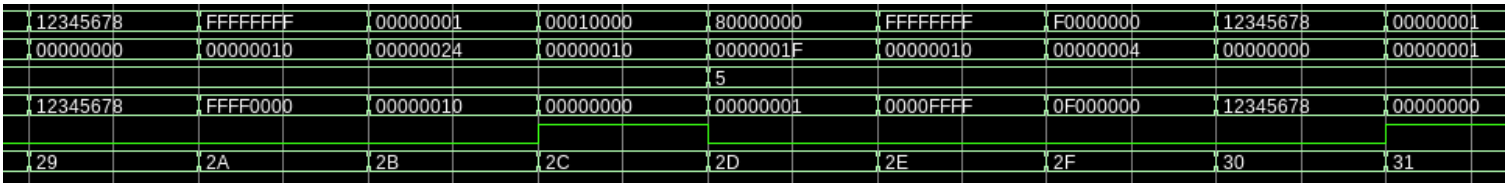












Each test case is numbered and has a corresponding number on the waveform to help identify the case, but they can also be determined based on the inputs and control value

[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

We test:

- Writes to each register using xxx.s
- Basic functionality of add, addi, lw, sw, etc through xxx.s

Branching instructions, beq,bne, bge,blt,bgeu,bltu both true and false through xxx.s
Barrel shifter functionality, sll,srl,sra,slli,srli,srai with xxx.s

[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

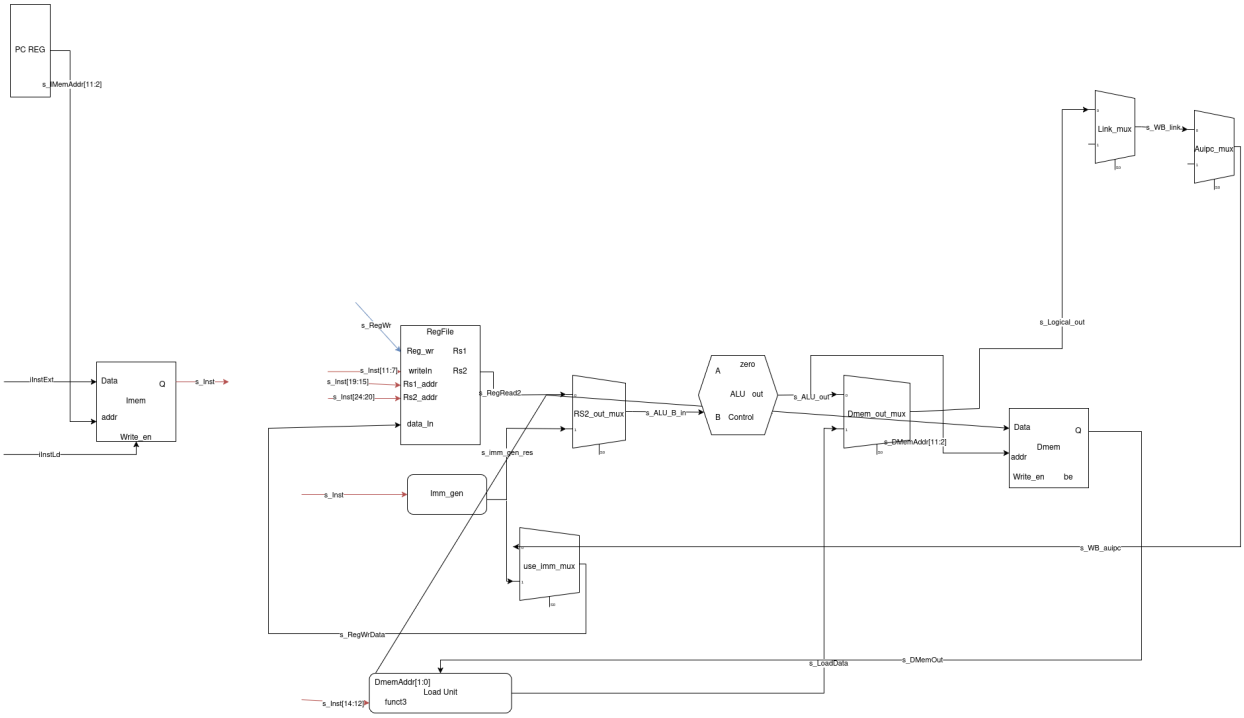
[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.

[Part 4.c] Create and test an application that sorts an array with N elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

Critical path is
PC->regfile->rs2->r2s_out_mux->ALU->DMEM_out_mux->dmem->regfile(wb)
Takes 47.014ns, max frequency is therefore 21.27 MhZ.



Biggest improvements can be made by shortening the mux path from dmem out to write back. Savings along the ALU will also help. Increasing speed of retrieval from IMEM and DMEM will also help but may not be possible.